



Security Review For Sodax



Collaborative Audit Prepared For:
Lead Security Expert(s):

Sodax
hildingr
TIMOH

Date Audited:

May 26 - May 29, 2026

Introduction

The LST Vault is a smart contract on Sonic that lets users deposit a liquid staking token and automatically earn an amplified version of the underlying staking yield. The vault achieves this by supplying the deposited asset as collateral into the SODAX money market, borrowing against it, converting the borrowed amount back into more of the same token, and repeating all without any manual steps from the user.

This audit covers the full set of contracts powering that strategy: the core ERC-4626 vault, the keeper-triggered rebalancing logic, and the hook contracts that execute leverage and deleverage steps atomically inside the SODAX intent system.

Scope

Repository: `icon-project/sodax-contracts`

Audited Commit: `9599accc7b0faf5eaeccec809835d6a40531ff81c`

Final Commit: `c5041fba88c97993717627e4682361b1a7eafdfa`

Files:

- `evm/contracts/intents/Intents.sol`
- `evm/contracts/vaults/LeverageYieldVault.sol`
- `evm/contracts/vaults/libraries/VaultValidation.sol`

Final Commit Hash

`c5041fba88c97993717627e4682361b1a7eafdfa`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	3	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Leverage intent can bypass min health factor [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/issues/5>

Summary

`_beforeLeverageUp()` calculations assume there is no slippage which is wrong.

Vulnerability Detail

There is slippage specified, i.e. leverage hook can supply less collateral than newly borrowed debt. Function `_leverageUp()` performs validation, checks that Intent created won't push risks beyond min health factor. Those calculations don't account for slippage, therefore overestimate final health:

```
function _leverageUp() internal {
    ...

    if (borrowAmount == 0) return;
    @> if (!_beforeLeverageUp(borrowAmount)) return;

    ...

    uint256 expectedAssetOut = _borrowTokenToAsset(borrowAmount);
    @> uint256 minAssetOutput = (expectedAssetOut * (MAX_BPS - slippageTolerance))
    ↪ / MAX_BPS;

    _createIntent(
        address(borrowToken),
        asset(),
        borrowAmount,
    @> minAssetOutput,
        leverageHook,
        leverageHook
    );
}

function _beforeLeverageUp(
    uint256 borrowAmount
) internal view returns (bool) {
    ...
    // LTV check in borrow-token units - ratio is valid regardless of USD price
    uint256 projDebtBT = _getDebtInBorrowToken() + borrowAmount;
    @> if ((projDebtBT * MAX_BPS) / (_assetToBorrowToken(collInAsset) +
    ↪ borrowAmount) > targetLTV + ltvTolerance)
```

```

        return false;

    // Use AAVE's oracle + its enforced LTV / liquidationThreshold so the
    ↪ projected
    // position matches exactly what validateBorrow will check at fill time.
    (uint256 totalCollBase, uint256 totalDebtBase, , uint256 liqThreshold,
    ↪ uint256 aaveLtv, ) =
        pool.getUserAccountData(address(this));

@>    uint256 projColl = totalCollBase + (_borrowTokenToAsset(borrowAmount) *
    ↪ aaveOracle.getAssetPrice(asset())) / 1e18;
    uint256 projDebt = totalDebtBase + (borrowAmount *
    ↪ aaveOracle.getAssetPrice(address(borrowToken))) / 1e18;

    // AAVE will reject the borrow with COLLATERAL_CANNOT_COVER_NEW_BORROW if
    // post-borrow LTV exceeds the user's weighted max LTV.
    if (projDebt * MAX_BPS > projColl * aaveLtv) return false;

    return (projColl * liqThreshold * 1e18) / (projDebt * MAX_BPS) >=
    ↪ minHealthFactor;
}

```

Impact

Contract can create intent which will increase exposure beyond specified limit.

Code Snippet

<https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/blob/main/sodax-contracts/evm/contracts/vaults/LeverageYieldVault.sol#L570-L597>

Tool Used

Manual Review

Recommendation

Account for slippage buffer in `_beforeLeverageUp()` checks:

```

function _beforeLeverageUp(
    uint256 borrowAmount
) internal view returns (bool) {
    if (pool.getReserveData(address(borrowToken)).currentVariableBorrowRate >
    ↪ maxBorrowRate)
        return false;
}

```

```

uint256 collInAsset = _getCollateralInAsset();
if (collInAsset == 0) return false;
+
+   uint256 borrowAmountAfterSlippage = borrowAmount * (MAX_BPS -
↪ slippageTolerance) / MAX_BPS;

    // LTV check in borrow-token units - ratio is valid regardless of USD price
    uint256 projDebtBT = _getDebtInBorrowToken() + borrowAmount;
-   if ((projDebtBT * MAX_BPS) / (_assetToBorrowToken(collInAsset) +
↪ borrowAmount) > targetLTV + ltvTolerance)
+   if ((projDebtBT * MAX_BPS) / (_assetToBorrowToken(collInAsset) +
↪ borrowAmountAfterSlippage) > targetLTV + ltvTolerance)
    return false;

    // Use AAVE's oracle + its enforced LTV / liquidationThreshold so the
    ↪ projected
    // position matches exactly what validateBorrow will check at fill time.
    (uint256 totalCollBase, uint256 totalDebtBase, , uint256 liqThreshold,
    ↪ uint256 aaveLtv, ) =
        pool.getUserAccountData(address(this));

-   uint256 projColl = totalCollBase + (_borrowTokenToAsset(borrowAmount) *
↪ aaveOracle.getAssetPrice(asset())) / 1e18;
+   uint256 projColl = totalCollBase +
    ↪ (_borrowTokenToAsset(borrowAmountAfterSlippage) *
    ↪ aaveOracle.getAssetPrice(asset())) / 1e18;
    uint256 projDebt = totalDebtBase + (borrowAmount *
    ↪ aaveOracle.getAssetPrice(address(borrowToken))) / 1e18;

    // AAVE will reject the borrow with COLLATERAL_CANNOT_COVER_NEW_BORROW if
    // post-borrow LTV exceeds the user's weighted max LTV.
    if (projDebt * MAX_BPS > projColl * aaveLtv) return false;

    return (projColl * liqThreshold * 1e18) / (projDebt * MAX_BPS) >=
    ↪ minHealthFactor;
}

```

Discussion

gcranju

Fix: <https://github.com/icon-project/sodax-contracts/pull/658>

`_beforeLeverageUp` now accounts for slippage on the collateral side. We compute `minCollateralAddedBT = borrowAmount * (MAX_BPS - slippageTolerance) / MAX_BPS` (which matches the `minOutputAmount` floor encoded in the intent) and use that in both the borrow-token LTV check and the AAVE projection. The debt side still uses the full

borrowAmount, so the projected health factor now reflects the actual worst-case post-fill state rather than an optimistic 1:1 swap.

Issue M-2: Withdrawal before Intent execution leads to health beyond min limit [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/issues/6>

Summary

Withdrawal between Intent creation and execution can result in final health lower than specified limit, potentially close to liquidation.

Vulnerability Detail

Withdraw operation withdraws collateral from position and transfers to user. Basically such operation increases leverage. Suppose following scenario:

- 1) Protocol needs rebalance to leverage up
- 2) Leverage up Intent is created
- 3) User performs withdraw. It reduces collateral and therefore increases leverage
- 4) Intent is executed, it pushes leverage up again. So now leverage is too high and position is close to liquidation.

Issue is possible because all health checks are performed at the moment of Intent creation, there are no checks at execution.

Impact

Withdrawal between Intent creation and execution can result in final health lower than specified limit, potentially close to liquidation.

Tool Used

Manual Review

Recommendation

Fix is not obvious. Consider adding health check during Intent execution. Permissioned ticks and solver will improve situation, monitor final health before Intent filling.

Discussion

gcranju

For now, we're going with permissioned ticks and a non-zero solver as recommended. Adding health checks during intent execution would require an architectural change to the hooks, so we're deferring that.

Fix: <https://github.com/icon-project/sodax-contracts/pull/657>

Issue M-3: Intent stacking can lead to DDOS of rebalance fills [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/issues/7>

Summary

Permissionless `tick()` allows an attacker to pre-emptively stack stale rebalance intents that consume the vault's approvals and DoS a later, more critical rebalance for up to 30 minutes.

Vulnerability Detail

`tick()` is exposed as a public, permissionless keeper entry point in [LeverageYieldVault.sol#L363-L365](#):

```
function tick() external nonReentrant {
    _rebalanceIfNeeded();
}
```

Each call to `tick()` that finds a sufficient LTV deviation triggers `_leverageUp()` or `_leverageDown()`, which in turn calls `_createIntent(...)`. The intent is created with a 30-minute deadline and the vault grants the corresponding allowance / credit delegation to the leverage / deleverage hook.

Because nothing prevents multiple intents from existing simultaneously, and the solver field can legitimately be set to `address(0)` via `setSolver()`, any party can fulfill the intent when the solver is unset.

Impact

Attack scenario (DoS via stale intent stacking):

1. The contract is in a state where a small de-leverage intent can already be created (LTV slightly above `targetLTV + ltvTolerance`), and `solver == address(0)` so anyone can fill intents.
2. An attacker spams `tick()` multiple times in a short window (e.g. 5 calls in 5 minutes), each producing an intent that consumes part of the vault's credit-delegation/`aToken` allowance to the hook. Only one of these is needed to actually move the LTV; the rest sit as "stale" but still-valid intents for up to 30 minutes.
3. The market subsequently shifts (e.g. a large price move that has not yet been reflected on-chain by the Chainlink feed). When the new price is reflected and a

much larger rebalance is needed, the next `tick()` creates a new, larger intent and re-approves the hook for the larger amount.

4. The attacker (or any third party) now fills one of the old, smaller intents first. Filling consumes the allowance/credit-delegation that was just re-set for the larger intent (since the hook contract pulls from the same approval pool), leaving the new intent under-collateralized in approval terms and therefore unfillable.
5. The attacker repeats, and for the next ~25 to 30 minutes the vault cannot rebalance to the size actually demanded by the market, because every freshly-created larger intent is "consumed" by a stale smaller one.

A more directed variant: the attacker watches the off-chain market, predicts that the on-chain price update will require an aggressive de-leverage, and pre-emptively spams `tick()` immediately before the oracle updates. The pre-created small intents then block every larger intent that the protocol tries to create during the 30-minute window.

Recommendation

Make `tick()` permissioned, e.g. restrict it to a whitelisted set of keeper addresses, so an unauthenticated party cannot drive intent creation.

Discussion

gcranju

Fix: <https://github.com/icon-project/sodax-contracts/pull/657>

`tick()` is now gated by `onlyKeeper` (owner + whitelisted keepers via `addKeeper` / `removeKeeper`), so unauthenticated parties can no longer drive intent creation. Additionally, `setSolver` now rejects `address(0)`, closing the "solver unset → anyone can fill" half of the attack – stale intents created during a legitimate keeper tick can't be opportunistically filled by an attacker either.

Issue L-1: LeverageYieldVault.maxRedeem() is not compliant with ERC4626 [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/issues/4>

Summary

LeverageYieldVault does not override function `maxRedeem()`

Vulnerability Detail

There is limit on how much user can withdraw from vault. That's because asset is collateral on leveraged position, so withdrawal reduces health factor. There is function `maxWithdraw()` that calculates correctly max allowed amount, but there is also function `maxRedeem()` which for the time being returns default value from OZ implementation:

```
function maxRedeem(address owner) public view virtual returns (uint256) {  
    return balanceOf(owner);  
}
```

Impact

Contract is not compliant with ERC4626

Code Snippet

<https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/blob/main/sodax-contracts/evm/contracts/vaults/LeverageYieldVault.sol#L691-L712>

Tool Used

Manual Review

Recommendation

Override `maxRedeem()` to handle `minHealthFactor` limit like in function `maxWithdraw()`

Discussion

gcranju

Fix: <https://github.com/icon-project/sodax-contracts/pull/658> maxRedeem is now overridden to be HF-aware, mirroring maxWithdraw.

Issue L-2: Already created can be filled even if they should be rejected due to LTV target updates [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/issues/8>

Summary

setTargetLTV and setLTVTolerance only update the storage variables; they do not invalidate intents that were already created under the previous LTV / tolerance values. Until the owner-drained transition is complete, a stale intent can be filled and move the vault into a position that the new parameters were specifically configured to disallow.

Vulnerability Detail

In `LeverageYieldVault.sol` #L868-L884:

```
function setTargetLTV(uint256 _targetLTV) external onlyOwner {
    VaultValidation.validateLtvAgainstPool(pool, asset(), eModeCategory,
        ↪ _targetLTV, ltvTolerance);
    uint256 old = targetLTV;
    targetLTV = _targetLTV;
    emit TargetLTVUpdated(old, _targetLTV);
}

function setLTVTolerance(uint256 _ltvTolerance) external onlyOwner {
    VaultValidation.validateLtvAgainstPool(pool, asset(), eModeCategory, targetLTV,
        ↪ _ltvTolerance);
    uint256 old = ltvTolerance;
    ltvTolerance = _ltvTolerance;
    emit LTVToleranceUpdated(old, _ltvTolerance);
}
```

LTV / tolerance bounds are only consulted when an intent is *created* – `_leverageUp` / `_leverageDown` compute amounts from the values current at `_createIntent` time, and `_beforeLeverageUp` checks `targetLTV + ltvTolerance` at the same moment. Once the intent exists, fill-time logic does not re-read these values, and the intent carries its own 30-minute deadline.

Scenario:

1. Vault has a pending lever-up intent created with `targetLTV = 7000`, `ltvTolerance = 300`.

2. Owner observes worsening market conditions and calls `setTargetLTV(5000)` (or tightens `ltvTolerance`) to make the position more conservative.
3. The pre-existing intent is still valid for up to 30 minutes and can be filled by a solver. The fill borrows the amount that was sized for the old, looser bounds, pushing the vault to an LTV the new configuration was meant to forbid.

Recommendation

Either enforce on-chain such that intents are blocked if they would violate the new parameters, or document that owner must drain pending intents or wait before updating these parameters.

Discussion

gcranju

Fix: <https://github.com/icon-project/sodax-contracts/pull/657> Going with second of the recommendation. We've shortened the intent deadline from 30 minutes to 5 minutes and documented the operational caveat directly on `setTargetLTV` / `setLTVTolerance`: the owner should wait for any in-flight rebalance intents to expire (≈5 min) before tightening these parameters, since existing intents are sized against the previous bounds and remain fillable until their deadline.

On-chain re-validation at fill time would require an architectural change to the hooks, same reasoning as #6.

TIMOH593

@gcranju `_createIntent()` still uses 30 min instead of 5

gcranju

@gcranju `_createIntent()` still uses 30 min instead of 5

<https://github.com/icon-project/sodax-contracts/pull/659>

This is the fix PR for this, I missed to send this, sorry for inconvenience!

Issue L-3: Setting solver to address(0) does not block intent filling as comment suggests [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-05-sodax-hana-wallet-may-25th-2026/issues/9>

Summary

The comment in `LeverageYieldVault.setSolver` claims that passing `address(0)` "temporarily disables intent filling." In practice, setting `solver = address(0)` makes every new intent permissionlessly fillable by any caller, the opposite of what the comment states.

Vulnerability Detail

In `LeverageYieldVault.setSolver`:

```
function setSolver(address _solver) external onlyOwner {
    // Allow address(0) to temporarily disable intent filling.
    address old = solver;
    solver = _solver;
    emit SolverUpdated(old, _solver);
}
```

`Intents.fillIntent` treats `intent.solver == address(0)` as "no restriction":

```
if (intent.solver != address(0) && intent.solver != msg.sender) revert
    ↳ InvalidSolver();
```

So `setSolver(0)` doesn't disable fills, it opens them to anyone.

Recommendation

Fix the comment to reflect actual behavior, or reject `address(0)` in `setSolver` if disabling fills was the intent.

Discussion

gcranju

Fix: <https://github.com/icon-project/sodax-contracts/pull/657>

`setSolver` now requires a non-zero address (`require(_solver != address(0), "Invalid solver")`), so the misleading "temporarily disable intent filling" path is removed entirely.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.